

DECISIONES DE DESPLIEGUE EN LA NUBE

Contexto General

Carpeta Ciudadana es un sistema gubernamental que necesita manejar documentos digitales de ciudadanos con alta seguridad, disponibilidad del 99.9% y capacidad de escalar desde pocos usuarios hasta millones. El sistema debe cumplir con leyes colombianas de protección de datos y mantener trazabilidad completa de todas las operaciones.

Criterios que Usamos para Decidir

Para cada componente evaluamos seis aspectos principales. La escalabilidad y disponibilidad tienen el mayor peso con 25% cada una, porque el sistema debe funcionar siempre y crecer con la demanda. La seguridad representa el 20% porque manejamos datos sensibles del gobierno. El rendimiento es 15% porque los ciudadanos esperan respuestas rápidas. El costo es solo 10% porque la prioridad es que el sistema funcione bien, aunque obviamente no se puede desperdiciar plata de más. La mantenibilidad es 5% porque aunque importante, los otros factores pesan más.

Decisión Principal: Todo en la Nube de Azure

Decidimos desplegar absolutamente todo en Azure, sin componentes on-premise. Esto ya que si se quiere tener un servicio disponible casi 24/7 on-premise, costaría mucho mantenerlo, y más aún costear resiliencia comprando un segundo datacenter, y si se quiere escalar, es poco probable que esto suceda debido a las limitaciones propias de la estructura.

Microservicios en Kubernetes (AKS)

Todos nuestros servicios backend corren en Azure Kubernetes Service. Evaluamos otras opciones como Azure App Service que es más simple, pero nos limitaba en el control de red y es más costoso de mantener a largo plazo. También consideramos usar máquinas virtuales directamente pero requeriría que nosotros manejemos toda la orquestación manualmente o tener que encargarnos de organizar mejor la orquestación con la integración de herramientas de kubernetes sin configuraciones previas.

Kubernetes nos da 3 beneficios importantes: Primero, escala automáticamente. Cuando llegan muchos documentos a procesar, el sistema crea hasta 30 trabajadores automáticamente y luego los destruye cuando ya no se necesitan. Esto lo hace en menos de 60 segundos. Segundo, tiene autocuración. Si un servicio falla, Kubernetes lo reinicia automáticamente sin intervención humana. Tercero, distribuye los servicios en tres zonas de disponibilidad diferentes, así que si una zona falla, las otras dos siguen funcionando.

Configuramos tres grupos de nodos. El grupo system tiene máquinas pequeñas y solo corre componentes críticos del cluster. El grupo user tiene máquinas medianas y corre nuestros servicios principales con escalado automático de 3 a 10 instancias. El grupo spot usa máquinas que Azure puede quitar en cualquier momento pero cuestan 60% menos, perfecto para trabajos que pueden reiniciarse si se interrumpen.

Bases de Datos como Servicio

Para PostgreSQL y Redis para el caché decidimos usar los servicios administrados de Azure en lugar de instalarlos nosotros en máquinas virtuales. Se usó Azure Database for PostgreSQL porque provee alta disponibilidad automática con réplicas en diferentes zonas, backups automáticos cada día, y puede escalar verticalmente hasta 64 cores sin downtime. Si se tuviera un servicio local de PostgreSQL, este mismo sería más difícil de mantener a largo plazo ya que no soportaría una gran cantidad de peticiones como cuando está en la nube.

Redis da replicación automática, persistencia cada 15 minutos, y puede manejar 10 mil conexiones simultáneas. Lo usamos para cuatro cosas principales: limitar cuántas peticiones puede hacer un usuario por minuto según su plan, guardar sesiones de usuario, mantener el estado de los circuit breakers que protegen al sistema cuando servicios externos fallan, y para locks distribuidos que evitan que dos trabajadores procesen el mismo documento al mismo tiempo.

Almacenamiento de Documentos

Azure Blob Storage es donde guardamos todos los PDFs y archivos. Decidimos usarlo porque tiene una característica llamada WORM que significa Write Once Read Many. Una vez que firmamos un documento, el sistema lo marca como inmutable y nadie puede modificarlo ni borrarlo, ni siquiera nosotros como administradores. Esto cumple con las leyes de archivo gubernamental.

Azure garantiza once nueves de durabilidad, que significa que de cada 100 mil millones de archivos, solo perderías uno. El mismo no es viable en on-premise ya que el costo de mantenerlo es mucho más elevado y el costo de tenerlo en la nube es bastante bajo.

Cuando un ciudadano sube un documento, generamos una URL temporal que solo funciona 15 minutos y le permite subir directamente a Blob Storage sin pasar por nuestros servidores. Esto ahorra ancho de banda y es más rápido. Para documentos firmados, aplicamos la política de inmutabilidad que los protege legalmente.

Sistema de Mensajería

Azure Service Bus maneja todas las comunicaciones asíncronas entre servicios. Evaluamos RabbitMQ que podríamos instalar en Kubernetes, pero requeriría que nosotros manejemos el cluster, hagamos respaldos y monitoreemos su salud además de que Service Bus se integra perfectamente con los demás servicios Azure de nuestra app.

Seguridad con Key Vault

Azure Key Vault guarda todas las contraseñas, claves de API y secretos. Configuramos rotación automática cada 90 días y usamos External Secrets Operator en Kubernetes para que cuando una clave rota, automáticamente se actualiza en todos los servicios sin reiniciarlos.

Nunca escribimos secretos en el código ni en archivos de configuración. Todo se lee dinámicamente desde Key Vault usando Workload Identity, que es una forma de que los pods de Kubernetes se autenticuen sin necesitar contraseñas. Key Vault registra quién accedió qué secreto y cuándo, dándonos auditoría completa.

Tráfico Entrante con Azure Load Balancer

Decidimos usar Azure Load Balancer en lugar de Azure Front Door como punto de entrada principal porque es más simple y económico. Load Balancer cuesta 60% menos que Front Door y cumple nuestras necesidades sin agregar complejidad innecesaria. Como nuestro público objetivo está concentrado en Colombia, no necesitamos CDN global ni enrutamiento geográfico avanzado. La latencia con Load Balancer es menor a 50 milisegundos para usuarios colombianos.

Load Balancer se integra nativamente con AKS, lo que significa que cuando creamos un servicio de tipo LoadBalancer en Kubernetes, Azure automáticamente provisiona el balanceador, configura health probes hacia los pods, y actualiza los backend pools cuando escalamos. Esto reduce significativamente la complejidad operativa comparado con Front Door que requeriría configuración manual de orígenes y reglas de enrutamiento.

Hub MinTIC y Resiliencia

El Hub MinTIC es un sistema externo gubernamental que corre en Heroku en Estados Unidos y no lo controlamos. Implementamos un patrón Circuit Breaker que monitorea la salud del Hub. Si 5 peticiones consecutivas fallan, el Circuit Breaker se abre y durante 5 minutos no enviamos más peticiones para darle tiempo al Hub de recuperarse.

Cuando el Circuit Breaker está abierto, los ciudadanos se registran localmente de todas formas pero quedan marcados como pendientes de sincronización. Un worker automáticamente reintenta cada 5 minutos hasta que el Hub vuelve. Los operadores también pueden sincronizar manualmente desde el panel de administración.

Esta arquitectura nos hace resilientes a fallos externos. Si el Hub MinTIC está caído, nuestro sistema sigue funcionando para todo lo demás y se sincroniza después automáticamente.

Cumplimiento Normativo

La configuración cumple con Ley de Habeas Data porque todos los datos están físicamente en la región Colombia Central de Azure. Las regulaciones de archivo gubernamental requieren retener documentos oficiales por 5 años. La política WORM de Blob Storage hace esto cumplimiento obligatorio a nivel técnico, no solo de proceso.